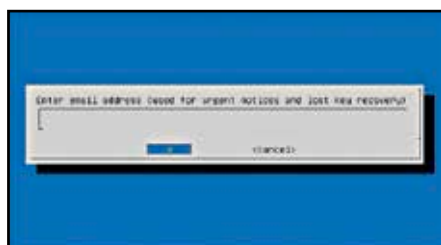


You're passing the arguments '--apache -d yourdomainname.com' to letsencrypt. The '--apache' tells Certbot which web server you are using and the domain you want to issue your certificate for is passed after the '-d' flag. In this case, it would be yourdomainname.com.

Should you want a Multi-Domain or Wildcard SSL certificate then you can add an additional domain by using another '-d' flag followed by the second domain name. For example:

```
sudo letsencrypt --apache -d yourdomainname.com -d yourseconddomain.com
```

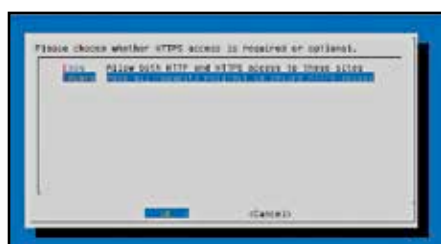
It's best to pass the root domain first before passing any sub-domains or wildcards as it's the domain used by Let's Encrypt to create the certificate. You'll then have to enter your email ID in the next prompt for key recovery should you forget it.



The rest is a pretty simple process where you just have to confirm if all the information on the screen is true or not.



Towards the end, you'll be asked if you want to route even the HTTP traffic through HTTPS which is ideally the safest option.



*pointers

>>Interesting developer videos



*Secure Nginx

>>This tutorial shows you how to use Let's Encrypt to obtain a free SSL certificate and use it with Nginx on Ubuntu 14.04.

<http://dgit.in/NginxUbuntu>



*Secure Wordpress

>> Learn how to put a WordPress site under SSL, removing "mixed content" warnings, and more.

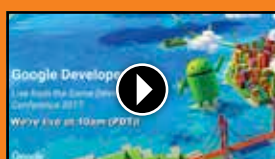
<http://dgit.in/SecureWrdPress>



*Facebook F8 2017

>>In case you missed it, check out the entire Facebook developer conference streamed live by Time in this video.

<http://dgit.in/FacebkF8>



*Google Dev Day

>>At GDC 2017, Developers talk about new devices, platforms, and tools are helping developers grow successful businesses and push the limits of mobile gaming.

<http://dgit.in/GDCDevDay>

Congratulations, you now have HTTPS set up for your domain. Your certificates will be located in '/etc/letsencrypt/live'. You can see if it's routing all traffic by typing `http://yourdomainname.com` and checking if the padlock symbol appears. Or you can check out ssllabs.com which has an SSL test page.



Now that you've got your SSL certificate, you need to set up auto-renewal since the certificate expires in 90 days. To do this, Let's Encrypt has a renew command that automatically checks and renews your certificate if it detects an expiry set to happen under 30 days. So go ahead and type the following in shell.

```
sudo letsencrypt renew
```

You'll be met with the following message since you've just installed the HTTPS certificate and have 90 days left.

- Processing /etc/letsencrypt/renewal/yourdomainname.com.conf
- The following certs are not due for renewal yet:
- /etc/letsencrypt/live/yourdomainname.com/fullchain.pem (skipped)
- No renewals were attempted.

Now we just have to set up a scheduled task to run every week or day or month which runs the renew command. The safest option would be to run it every day but we feel that's a little unnecessary at this point. So we're going to be running it once a week.

You'll need to edit the crontab to set up a schedule. Type the following command to open it.

```
sudo crontab -e
```

Based on which text editors are installed on your system, you'll be presented with options to pick one. Once crontab is open, you'll need to add an additional line at the very end for your SSL renewal. Crontab follows a simple pattern which has your date and time first and the command to be executed right after that. For example:

```
59 23 31 12 6 command
```